

Week 1 - Plan

- Introduction
- Video recording, code will be shared
- About Git and GitHub
- Write code using Copilot
- Strengths and Weaknesses of LLMs
- IDE and browser tiled on screen
- VSCode installation check, opening terminal
 - In terminal, check
 - node, npm
 - git
 - Config

```
git config --global user.name "Your name"
git config --global user.email "your_email@domain.com"
```

- creating GitHub account check
- GitHub Copilot Chat extension check installed and Enabled
 - Sign in to GitHub
 - online
 - In VSCode
 - Verify Business plan activated - invitation from GL to be accepted to see this
 - Copilot settings on github.com
 - Copilot dialogs overview
- Install Live Server extension (Ritwick Dey)
- About the ProConnect project
 - Download from Week 1 files and open the folder (make sure to open the exact folder)
 - Open app in live server
 - Understand the project
 - Understand the files
 - **NOTE:** `misc.js` is unrelated to the project
- About Git and GitHub
 - Overview
 - Init the project as a Git project and add, commit, push
 - All through VSCode UI
- How AI tools, LLMs work in general
 - Prompt + word prediction
 - Tokenizer
 - <https://platform.openai.com/tokenizer>
 - Even code is tokenized
- Start with exercises in `misc.js`
 - Remember to save files after you edit in VSCode
 - Copilot inline suggestion
 - **NOTE:**
 - Accept suggestions with TAB or partially accept with Cmd/Ctrl + Tab

- Comment - Function to calculate factorial of a number
 - Comment - Filter even numbers in `numbers`
 - Notice code completion uses file contents as "context"
 - Edit suggestions / partially accept it and continue coding
- Select existing code and edit it
 - Change variable name to see Next Edit Suggestions
- Opening Copilot chat window and understanding it
- Model benchmarking results
 - <https://arena.ai/leaderboard>
- **IMPORTANT:** Keep track of usage and open new session windows for unrelated questions to avoid long context issues
- Ask mode
 - As a Java developer, learn JS
 - As a Excel developer, learn JS
 - Give exact expectations on what you want to learn
 - Code explanation
 - Ask for line by line explanation of a code snippet
 - Ask for high level explanation of a code snippet
 - Code improvement
 - Ask for code improvement suggestions
 - Ask for security improvement suggestions
 - Ask for performance improvement suggestions
 - Codebase explanation
 - Code translation
 - Translate code from one language to another
 - Selecting more files as context
 - Select `index.js` and `index.html` and ask for an explanation of how the code works together
- Inline chat:
 - Cmd + I
 - In `misc.js`
 - Paste code snippet (Example 4) and ask for explanation
 - Ask for Refactoring using modern JS
 - Ask for adding input validation and error handling
 - In `index.js`
 - Ask to add like button functionality to the existing code
- Commit changes with meaningful commit messages
- Now switch to "Agent" mode
 - Be descriptive in prompts to get better results. Set role. Set expctations clearly.
 - Explicitly add files as context if needed. Don't rely on the model to pick the right context from the entire codebase.
 - For function / features mention input and output expectations clearly.
 - EXERCISE: Prompt "Add a sidebar to the app with navigation links to Home, Profile, Settings. Highlight the active link based on the current page. Fix the sidebar on the left of the screen and make it responsive."
 - Add HTML, CSS, JS file to the context and ask for implementation
 - Review the code and ask for improvements
 - Commit code and check the commit history on Github